

PROGRAMMING FOR PROBLEM SOLVING [R22]

UNIT 1

SHORT ANSWER QUESTIONS

1. Define a) Variable b) Identifier c) Constant
2. Write short notes on number system with examples
3. Write short notes on jump statements?
4. Define Flowchart and Algorithm
5. Write short notes on Continue statements?

LONG ANSWER QUESTIONS

1. Write a Short notes on Creating and Running the program?
2. Write about Different data types along with memory?
3. Write about Decision making statements with example?
4. Write a Program on sum of n numbers using While and for loop?
5. Describe about computer system

UNIT 2

SHORT ANSWER QUESTIONS

1. Write short notes on pointer Arithmetic Operations?
2. Define Array?
3. Define 1) Array of pointers 2) Pointers to functions.
4. Define a String with a example?
5. Discuss Difference between Structure and Unions

LONG ANSWER QUESTIONS

1. Write about String Input / Output functions with example?
2. Write a C Program to reverse a given string with and without using functions?
3. What are the types of Arrays? Explain in detail
4. Write a C program that reads 15 names each of up to 30 characters, stores them in an array, and uses an array of pointers to display them in ascending (ie. alphabetical) order
5. What is Self referral Structure? Explain with example?

UNIT 3

SHORT ANSWER QUESTIONS

1. Write Short notes on Preprocessor command 'define'
2. Difference between undef and ifndef.
3. Write the Difference between Binary File and Text File?
4. Define different types of file input output functions?
5. Define different file modes of operation?

LONG ANSWER QUESTIONS

1. What are Preprocessor commands? List some Preprocessor commands with the example?
2. Define a macro that finds the maximum of two numbers. Write a C program that uses the macro and prints the maximum of two numbers.
3. Write about Positioning functions (fseek ,rewind and ftell)
4. Write a C program to count the number of times a character occurs in a text file. The file name and the character are supplied as command-line arguments.
5. Write about file status functions (error handling) with example?

UNIT 4

SHORT ANSWER QUESTIONS

1. What is a Function? Write about signature of a function?
2. Write about limitation of recursion?
3. Explain passing parameters to functions.
4. Explain passing pointers to functions.
5. Describe the allocation of memory for arrays of different data types

LONG ANSWER QUESTIONS

1. Write about standard functions with examples?
2. Explain about memory allocation functions with example.
3. What is recursion? Write a program for factorial using recursion?
4. Difference between call by value and call by reference.
5. Write a recursive function for Fibonacci series.

UNIT 5

SHORT ANSWER QUESTIONS

1. Define order of complexity.
2. What is an algorithm and what are its advantages?
3. Write an algorithm to check whether a given number is prime or not.
4. Differences between linear search and binary search.
5. Compare different sorting algorithms.

LONG ANSWER QUESTIONS

- 1) Write algorithm to demonstrate binary search.
- 2) Explain the algorithm for finding roots of a quadratic equations
- 3) Define algorithm for finding minimum and maximum numbers of a given set
- 4) Explain Bubble sort algorithm with its order of complexity
- 5) Explain the techniques of Selection sort and Insertion sort with the help of an example array.

Unit 1

1. Define a) Variable b) Identifier c) Constant

Variable:

A variable in the C programming language is a named storage location that holds a value of a specific data type. To declare a variable in C, you need to specify the data type, followed by the variable name, and optionally assign a value to it.

For example:

```
int age = 30;
```

```
float temperature = 72.5;
```

```
char grade = 'A';
```

In this example, `age` is an integer variable with a value of 30,

`temperature` is a floating-point variable with a value of 72.5,

and `grade` is a character variable with a value of 'A'.

Identifier

An identifier in C is a name used to identify a variable, function, or any other user-defined item. An identifier can be composed of letters, digits, and underscores, and must start with either a letter or an underscore.

In C, there are some reserved keywords, such as `int`, `float`, `char`, `if`, etc., that cannot be used as identifiers. Additionally, identifiers are case-sensitive, meaning `myVariable` and `myvariable` are considered different identifiers.

It is a good practice to choose descriptive and meaningful names for identifiers, to make your code more readable and understandable. For example, `age` is a better identifier for a variable storing a person's age than `a`.

Constant

A constant in C is a value that cannot be changed during the execution of a program. Constants are declared using the `const` keyword, followed by the data type and the identifier name, and an optional assignment of a value.

For example:

```
const float pi = 3.14159265;
```

```
const int daysInWeek = 7;
```

In this example, `pi` is a constant with the value of `3.14159265` that represents the mathematical constant π , and `daysInWeek` is a constant with the value of `7` that represents the number of days in a week.

By using constants in your code, you can make it more readable and maintainable, as the values that are unlikely to change can be stored in a constant, and you don't have to keep track of the values manually.

2. Write short notes on number system with examples

A number system is a way of representing numbers using symbols or digits. There are several number systems, including:

- **Decimal (base-10) number system:** This is the most widely used number system, where each digit can be any value from 0 to 9. For example, the number 123 in decimal represents $1 * 100 + 2 * 10 + 3 * 1$.
- **Binary (base-2) number system:** This number system uses only two digits, 0 and 1. For example, the number 1101 in binary represents $1 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0$.
- **Octal (base-8) number system:** This number system uses eight digits, from 0 to 7. For example, the number 75 in octal represents $7 * 8^1 + 5 * 8^0$.
- **Hexadecimal (base-16) number system:** This number system uses sixteen digits, from 0 to 9 and A to F. For example, the number BEEF in hexadecimal represents $11 * 16^3 + 14 * 16^2 + 14 * 16^1 + 15 * 16^0$.

Each number system has its own advantages and disadvantages, and the choice of a number system often depends on the specific requirements of a particular application.

3. Write short notes on jump statements?

Jump statements in C are used to transfer control from one part of a program to another. The following are the most commonly used jump statements in C:

- **break:** This statement is used to exit a loop or switch statement. When a **break** statement is encountered inside a loop or switch, the loop or switch is immediately terminated, and the program control resumes at the next statement following the loop or switch.
- **continue:** This statement is used to skip the current iteration of a loop and move on to the next iteration. When a **continue** statement is encountered inside a loop, the current iteration of the loop is skipped, and the program control moves to the next iteration of the loop.
- **goto:** This statement is used to transfer control to a labeled statement in the same function. The **goto** statement is followed by a label and a colon, and the program control transfers to the statement marked with that label. The **goto** statement should be used with caution, as it can make the code hard to understand and maintain.
- **return:** This statement is used to exit a function and return a value to the calling function. The **return** statement is followed by an expression that specifies the value to be returned. The **return** statement terminates the execution of the function, and the program control returns to the calling function.

Jump statements are an important part of the control flow of a program and are used to change the normal sequence of execution. However, they should be used with caution, as they can make the code hard to understand and maintain.

4. Define Flowchart and Algorithm

A flowchart is a visual representation of the steps involved in a process, using symbols and arrows to show the flow of control from one step to another. Flowcharts are used to represent algorithms, processes, and systems, and are used to help understand and document complex procedures.

An algorithm, on the other hand, is a step-by-step procedure for solving a problem or performing a task. An algorithm is usually written in a formal

language, such as a programming language, and provides a detailed description of the steps required to solve a problem or perform a task.

The flowchart and algorithm are related, as the flowchart is often used to represent an algorithm. A flowchart provides a visual representation of an algorithm, making it easier to understand, while the algorithm provides a precise and detailed description of the steps involved in a process.

In summary, a flowchart provides a visual representation of a process, while an algorithm provides a step-by-step procedure for solving a problem or performing a task. Both flowcharts and algorithms are important tools for understanding and documenting complex procedures, and are widely used in fields such as computer science, engineering, and mathematics.

5. Write short notes on Continue statements?

The `continue` statement in C is a control flow statement that is used to skip the current iteration of a loop and move on to the next iteration. It is used inside the body of a loop, such as `for`, `while`, or `do-while`, and allows you to bypass the rest of the loop body for the current iteration.

When a `continue` statement is encountered, the program control immediately moves to the next iteration of the loop, bypassing any remaining statements in the current iteration. This allows you to skip an iteration under certain conditions and continue with the next iteration.

Here is an example of using a `continue` statement in a `for` loop:

```
for (int i = 0; i < 10; i++) {  
    if (i % 2 == 0) {  
        continue;  
    }  
    printf("%d\n", i);  
}
```

In this example, the `continue` statement skips the iteration of the loop if the value of `i` is even, and the `printf` statement is only executed for odd values of `i`.

The `continue` statement can be useful for simplifying complex loop conditions and making the code more readable. However, it should be used with caution, as overusing it can make the code harder to understand and maintain.

6. Write a Short notes on Creating and Running the program?

Creating and running a program involves several steps, including writing the code, compiling the code, and executing the compiled code. The following is a general outline of the steps involved in creating and running a program:

1. Write the code: The first step in creating a program is writing the code. You can use a text editor, such as Notepad or Sublime Text, to write the code. The code should be written in a programming language, such as C, Java, Python, etc.
2. Compile the code: Once you have written the code, you need to compile it. The compiler converts the code into machine-readable code, called an executable file. Compilation is necessary because computers cannot understand the code written in a programming language.
3. Run the program: After the code has been compiled, you can run the program by executing the compiled code. When the program is run, the computer executes the instructions in the code, and you can see the output of the program.

The process of creating and running a program may vary depending on the operating system and programming language being used. However, these are the general steps involved in creating and running a program.

In summary, creating and running a program involves writing the code, compiling the code, and executing the compiled code. These steps are necessary to turn the code written in a programming language into a working program that can be run on a computer.

7. Write about Different data types along with memory?

Data types in programming refer to the type of data that a variable can store. Different programming languages support different data types, but some common data types include integers, floating-point numbers, characters, and strings.

Here is an overview of some of the most commonly used data types, along with the amount of memory they typically consume:

1. Integer: An integer is a whole number that can be positive, negative, or zero. Integers typically use 4 bytes of memory, or 32 bits, and can store values in the range of -2,147,483,648 to 2,147,483,647.
2. Floating-point: A floating-point number is a number with a fractional component, such as 3.14 or -2.0. Floating-point numbers typically use 8 bytes of memory, or 64 bits, and can store much larger and smaller values than integers, with a high degree of precision.
3. Character: A character is a single letter, digit, or symbol. Characters typically use 1 byte of memory, or 8 bits, and can be stored in a variable of type `char`.
4. String: A string is a sequence of characters, and is used to represent text. Strings typically use multiple bytes of memory, depending on the length of the string and the character encoding used.
5. Boolean: A boolean is a data type that can only take on two values, either `true` or `false`. Booleans typically use 1 byte of memory, or 8 bits.

These are just a few of the data types that are commonly used in programming. The amount of memory used by each data type can vary depending on the specific implementation and the architecture of the computer being used.

In summary, data types in programming define the type of data that a variable can store, and the amount of memory used by each data type can vary depending on the specific implementation and the architecture of the computer being used. Common data types include integers, floating-point numbers, characters, strings, and booleans.

8. Write about Decision making statements with example?

Decision making statements in C allow a program to make decisions based on conditions and execute different parts of the code accordingly. They are used to control the flow of the program based on conditions, and provide the ability to perform different actions depending on the outcome of a condition.

There are several types of decision making statements in C, including `if`, `if-else`, `switch`, and `ternary operator`.

1. `if` statement: The `if` statement allows a program to execute a block of code only if a certain condition is true. Here is an example of an `if` statement in C:

```
int x = 10;
```

```
if (x > 5) {  
    printf("x is greater than 5\n");  
}
```

In this example, the `if` statement checks if the value of `x` is greater than 5, and if the condition is true, the program will print "x is greater than 5".

2. `if-else` statement: The `if-else` statement allows a program to execute a block of code if a certain condition is true, and a different block of code if the condition is false. Here is an example of an `if-else` statement in C:

```
int x = 10;  
if (x > 5) {  
    printf("x is greater than 5\n");  
} else {  
    printf("x is not greater than 5\n");  
}
```

In this example, the `if-else` statement checks if the value of `x` is greater than 5, and if the condition is true, the program will print "x is greater than 5". If the condition is false, the program will print "x is not greater than 5".

3. `switch` statement: The `switch` statement is used to execute one of several blocks of code based on a specific value. Here is an example of a `switch` statement in C:

```
int x = 2;  
switch (x) {  
    case 1:  
        printf("x is 1\n");  
        break;  
    case 2:  
        printf("x is 2\n");  
        break;  
    case 3:
```

```

    printf("x is 3\n");
    break;
default:
    printf("x is not 1, 2, or 3\n");
    break;
}

```

In this example, the `switch` statement checks the value of `x`, and based on the value of `x`, the program will execute one of the cases and print the corresponding message.

4. Ternary operator: The ternary operator is a shorthand way to write a simple `if-else` statement. It has the form `condition ? expression1 : expression2`. Here is an example of a ternary operator in C:

```

int x = 10;

int result = (x > 5) ? 100 : 200;

printf("Result: %d\n", result);

```

In this example, the ternary operator checks if the value of `x` is greater than 5, and if the condition is true, the value of `result` is set to 100, otherwise it is set to 200.

In summary, decision making statements in C provide a way for a program to make decisions based on conditions, and execute different parts of the code accordingly. Some common types of decision making statements include `if`, `if-else`, `switch`, and `ternary operator`.

9. Write a Program on sum of n numbers using While and for loop?

Here's an example of a program to find the sum of `n` numbers using both a `while` loop and a `for` loop in C:

```

#include <stdio.h>

int main() {
    int n, i, sum = 0;

    printf("Enter the value of n: ");
}

```

```

scanf("%d", &n);

// Using while loop
i = 1;
while (i <= n) {
    sum += i;
    i++;
}
printf("Sum of first %d numbers (using while loop): %d\n", n, sum);

// Using for loop
sum = 0;
for (i = 1; i <= n; i++) {
    sum += i;
}
printf("Sum of first %d numbers (using for loop): %d\n", n, sum);
return 0;
}

```

In this program, the user is prompted to enter the value of `n`, which represents the number of numbers to be summed up. The program then uses a `while` loop to find the sum of the first `n` numbers and a `for` loop to find the same sum. The sum of the first `n` numbers can be found using the formula $n * (n + 1) / 2$. The program calculates this sum and displays the result for both the `while` loop and the `for` loop.

10. Describe about computer system

A computer system is a combination of hardware and software components that work together to perform a wide range of tasks.

Hardware components include the central processing unit (CPU), memory (RAM), storage (hard disk drive or solid-state drive), input/output devices (such as keyboard, mouse, and monitor), and various peripheral devices (such as printers, scanners, and speakers).

Software components include operating systems, application software, and system utilities. The operating system is the foundation of the computer system, controlling all hardware and software components and providing a user interface. Application software includes programs such as word processors, spreadsheets, and graphic design tools, which are used to perform specific tasks. System utilities are programs that help maintain and optimize the computer system.

A computer system can be used for a wide range of purposes, including personal productivity, entertainment, education, and scientific research. The speed and capacity of computer systems have increased dramatically in recent years, making it possible to perform complex tasks and store large amounts of data quickly and efficiently.

Overall, a computer system is an essential tool for many people and organizations, enabling them to perform tasks and manage information efficiently and effectively.

Unit 2

1. Write short notes on pointer Arithmetic Operations?

Pointer arithmetic operations are a set of operations that can be performed on pointers in the C programming language. These operations allow you to manipulate and perform arithmetic on memory addresses stored in pointers. The following are some of the most common pointer arithmetic operations in C:

1. Increment: You can increment a pointer by a positive integer value using the `++` operator. This increases the memory address stored in the pointer by the size of the data type it points to. For example, `ptr++` increments the value of `ptr` by the size of `int`.
2. Decrement: You can decrement a pointer by a positive integer value using the `--` operator. This decreases the memory address stored in the pointer by the size of the data type it points to. For example, `ptr--` decrements the value of `ptr` by the size of `int`.
3. Addition: You can add a positive integer value to a pointer using the `+` operator. This increases the memory address stored in the pointer by the size of the data type it points to multiplied by the value being added. For example, `ptr + 2` increments the value of `ptr` by 2 times the size of `int`.

4. Subtraction: You can subtract a positive integer value from a pointer using the `-` operator. This decreases the memory address stored in the pointer by the size of the data type it points to multiplied by the value being subtracted. For example, `ptr - 2` decrements the value of `ptr` by 2 times the size of `int`.
5. Difference: You can calculate the difference between two pointers using the `-` operator. The result is the number of elements between the two pointers, not the number of bytes. For example, `ptr2 - ptr1` calculates the number of elements between `ptr1` and `ptr2`.

It is important to note that pointer arithmetic operations are only valid for pointers of the same data type, and that the result of a pointer arithmetic operation is a pointer of the same data type as the original pointer. Additionally, pointer arithmetic should only be performed within the bounds of an array or dynamically allocated memory, as performing pointer arithmetic outside of these bounds can result in undefined behavior.

2. Define Array?

An array in C is a data structure that stores a collection of elements of the same data type, such as integers, characters, or floating-point numbers. Arrays are declared with a fixed size, and elements are accessed using an index that starts from zero.

For example, the following code declares an array `arr` of size 5, containing integers:

```
int arr[5];
```

Elements of the array can be accessed using the square bracket notation, such as `arr[0]`, `arr[1]`, `arr[2]`, etc. The size of the array can be determined using the `sizeof` operator, like so:

```
int size = sizeof(arr) / sizeof(arr[0]);
```

It's also possible to initialize an array at the time of declaration, like so:

```
int arr[] = {1, 2, 3, 4, 5};
```

Arrays in C can be passed as arguments to functions, returned from functions, and assigned to variables. When arrays are passed as arguments to functions, the address of the first element of the array is passed, allowing the function to access all elements of the array.

Arrays in C are a useful data structure for storing and manipulating large collections of data. However, once an array is declared with a fixed size, it cannot be resized. To overcome this limitation, dynamic arrays can be created using pointers and memory allocation functions, such as `malloc()` and `realloc()`

3. Define

1) Array of pointers

An array of pointers is an array that stores addresses of variables or memory locations, rather than their actual values. Each element in the array is a pointer that points to a specific memory location, which can store a value of a particular data type.

For example, consider the following code that declares an array of pointers to `int` values:

```
int a = 10;
int b = 20;
int c = 30;
int *ptr[3];
ptr[0] = &a;
ptr[1] = &b;
ptr[2] = &c;
```

Here, the array `ptr` is declared as an array of 3 pointers to `int` values, and each element of the array is assigned the address of a different `int` variable `a`, `b`, and `c`. The values stored at these memory locations can be accessed by dereferencing the pointers using the `*` operator, like so:

```
printf("%d\n", *ptr[0]);
printf("%d\n", *ptr[1]);
printf("%d\n", *ptr[2]);
```

Arrays of pointers are useful in many applications, such as when you want to store a collection of values of different data types, or when you want to work with an array of dynamically allocated memory locations. Arrays of pointers can also be passed as arguments to functions, allowing the

function to access and manipulate the values stored at the memory locations pointed to by the pointers.

2) Pointers to functions.

In C, pointers to functions (also known as function pointers) are variables that store the addresses of functions. Function pointers are declared by specifying the return type and the argument list of the function, preceded by an asterisk `*`.

For example, consider the following code that declares a function pointer `ptr` to a function that takes two `int` values as arguments and returns an `int` value:

```
int (*ptr)(int, int);
```

Once a function pointer is declared, it can be assigned the address of a function using the function's name, like so:

```
int add(int a, int b) {  
    return a + b;  
}
```

```
ptr = &add;
```

To call a function through a function pointer, you need to dereference the pointer using the `(*ptr)` notation, followed by the arguments in parentheses, like so:

```
int result = (*ptr)(10, 20);  
printf("%d\n", result);
```

Function pointers are useful in many applications, such as when you want to pass a function as an argument to another function, or when you want to store a collection of functions and dynamically choose which function to call based on certain conditions. Function pointers also play a key role in C's event-driven programming, allowing you to write flexible and reusable code

4. Define a String with a example?

A string in C is a sequence of characters, terminated by a special character called the null terminator (`'\0'`). Strings are often used to store and manipulate text data.

In C, strings can be represented using arrays of characters, with each element in the array representing a single character in the string. The last element in the array must always be the null terminator, to indicate the end of the string.

For example, consider the following code that declares and initializes a string:

```
char str[6] = "hello";
```

Here, the string "hello" is stored in an array of characters `str`, with each character occupying a separate element in the array. The last element in the array is the null terminator, `'\0'`, which indicates the end of the string.

You can access individual characters in a string by using an index, starting from 0 for the first character, like so:

```
printf("%c\n", str[0]);  
printf("%c\n", str[1]);  
printf("%c\n", str[2]);  
printf("%c\n", str[3]);  
printf("%c\n", str[4]);
```

You can also use the `%s` format specifier to print a string, like so:

```
printf("%s\n", str);
```

Strings are an important data structure in C and are used in many applications, such as when you want to manipulate and process text data, when you want to store user input, or when you want to write and read data to and from files. C provides a rich set of functions for working with strings, such as `strlen`, `strcpy`, `strcat`, and `strcmp`, among others.

5. Discuss Difference between Structure and Unions

	STRUCTUTE	UNION
Keyword	The keyword struct is used to define a structure	The keyword union is used to define a union
Size	When a variable is associated with a structure, the compiler allocate the memory for each member. The size of structure is greater than or	When a variable is associated with a union, the compiler allocates the memory by considering the size of the

	equal to the sum of sizes of its members.	largest memory. So, size of union is equal to the size of largest member.
Memory	Each member within a structure is assigned unique storage area of location .	Memory allocated is shared by individual members of union.
Value altering	Altering the value of a member will not affect other members of the structure.	Altering the value of any of the members will alter other member values.
Accessing members	Individual members can be accessed at a time	Only one member can be accessed at a time.
Initialization of members	Several members of a structure can be initialized at once.	Only the first member of a union can be initialized.

6. Write about String Input / Output functions with example?

In C, there are several string input and output functions that allow you to read and write strings from and to the standard input/output (usually the keyboard and the screen). Some of the most commonly used string input and output functions are:

1. `fgets()`: This function is used to read a line of text from the standard input (usually the keyboard) and store it in a string.

Example:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    printf("Enter a string: ");
    fgets(str, 100, stdin);
    printf("You entered: %s", str);
    return 0;
}
```

2. `gets()`: This function is used to read a line of text from the standard input (usually the keyboard) and store it in a string. It is similar to

`fgets()`, but it does not have a parameter to specify the maximum size of the input, so it can be dangerous to use as it can lead to buffer overflow.

Example:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    printf("Enter a string: ");
    gets(str);
    printf("You entered: %s", str);
    return 0;
}
```

3. `puts()`: This function is used to write a string to the standard output (usually the screen). It automatically appends a newline character to the end of the string.

Example:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    strcpy(str, "Hello World!");
    puts(str);
    return 0;
}
```

4. `printf()`: This function is used to write formatted output to the standard output (usually the screen). You can use various conversion specifiers to format the output, including the `%s` conversion specifier to print a string.

Example:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    strcpy(str, "Hello World!");
    printf("The string is: %s\n", str);
    return 0;
}
```

In summary, these string input and output functions allow you to read and write strings from and to the standard input and output in C, and they provide different ways to format and control the input and output of strings.

7. Write a C Program to reverse a given string with and without using functions?

Here's a program to reverse a given string without using a function:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    int len, i;
    printf("Enter a string: ");
    scanf("%s", str);
    len = strlen(str);
    printf("The reverse of the string is: ");
    for (i = len - 1; i >= 0; i--) {
        printf("%c", str[i]);
    }
}
```

```
printf("\n");  
return 0;  
}
```

And here's a program to reverse a given string using a function:

```
#include <stdio.h>  
#include <string.h>  
void reverse(char *str) {  
    int len, i;  
    len = strlen(str);  
    for (i = 0; i < len / 2; i++) {  
        char temp = str[i];  
        str[i] = str[len - i - 1];  
        str[len - i - 1] = temp;  
    }  
}  
  
int main() {  
    char str[100];  
    printf("Enter a string: ");  
    scanf("%s", str);  
    reverse(str);  
    printf("The reverse of the string is: %s\n", str);  
    return 0;  
}
```

In the first program, the string is reversed by iterating through the string from its end to its beginning and printing each character. In the second program, the string is reversed using a function called `reverse()`. The function takes a pointer to a string as an argument, and it reverses the string by swapping the characters in the first and last halves of the string.

8. What are the types of Arrays? Explain in detail

Arrays in C can be classified into the following types based on the number of dimensions and how the data is stored:

1. **One-dimensional arrays:** A one-dimensional array is a linear collection of elements stored in contiguous memory locations. Each element can be accessed using a single index. For example, `int arr[10];` declares an array `arr` with 10 elements of type `int`.
2. **Multi-dimensional arrays:** A multi-dimensional array is an array of arrays. It is used to store data in a more organized and meaningful way. For example, a two-dimensional array `int arr[3][4];` declares an array `arr` with 3 rows and 4 columns.
3. **Jagged arrays:** A jagged array is an array of arrays where the length of each row can be different. It is also known as an "array of arrays." For example, `int arr[3][];` declares a jagged array `arr` with 3 rows and the number of columns for each row can be different.
4. **Character arrays:** A character array is a one-dimensional array used to store a sequence of characters. The null character `'\0'` is used to terminate the string. For example, `char str[10] = "Hello";` declares a character array `str` with the value "Hello".
5. **Array of pointers:** An array of pointers is an array where each element is a pointer to a variable. For example, `int *arr[5];` declares an array of pointers to `int` variables with 5 elements.
6. **Array of structures:** An array of structures is an array where each element is a structure. For example, `struct student stu[10];` declares an array of structures `stu` with 10 elements.

It is important to note that arrays in C are stored in contiguous memory locations, so accessing elements in an array is fast and efficient. However, the size of an array must be specified at compile time and cannot be changed during runtime.

9. Write a C program that reads 15 names each of up to 30 characters, stores them in an array, and uses an array of pointers to display them in ascending (ie. alphabetical) order

Here is an example implementation of the program in C:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```

#define N 15

#define MAX_NAME_LENGTH 30

int compare_names(const void *a, const void *b) {
    return strcmp(*(char **)a, *(char **)b);
}

int main() {
    char names[N][MAX_NAME_LENGTH];
    char *names_ptrs[N];

    printf("Enter %d names, each up to %d characters:\n", N,
MAX_NAME_LENGTH);

    for (int i = 0; i < N; i++) {
        printf("Enter name %d: ", i + 1);
        scanf("%s", names[i]);
        names_ptrs[i] = names[i];
    }

    qsort(names_ptrs, N, sizeof(char *), compare_names);

    printf("\nNames in alphabetical order:\n");
    for (int i = 0; i < N; i++) {
        printf("%s\n", names_ptrs[i]);
    }

    return 0;
}

```

This program uses `scanf` to read the names into the `names` array, and then creates an array of pointers to the names using `names_ptrs`. The names are sorted using the `qsort` function, which takes a comparison function as an argument. The comparison function, `compare_names`, uses `strcmp` to compare two names and determine their order. Finally, the sorted names are displayed using a loop and the array of pointers

10.What is Self referral Structure? Explain with example?

A self-referential structure is a structure in computer programming that contains a pointer or reference to an instance of the same structure type. This allows for the creation of complex data structures, such as linked lists, trees, and graphs, where each node in the structure can point to other nodes of the same type.

Here is an example of a self-referential structure in C that represents a node in a linked list:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

In this example, `struct Node` contains an integer data field and a pointer `next` to another instance of `struct Node`. This allows for the creation of a linked list, where each node points to the next node in the list. The last node in the list would have its `next` field set to `NULL` to indicate the end of the list.

Self-referential structures can be used to create many different types of data structures, including linked lists, trees, and graphs, and are an essential tool for computer programmers.

Unit 3

1. Write Short notes on Preprocessor command 'define'

The `#define` preprocessor directive in C is used to define a symbolic constant, also known as a macro. The macro is a name that represents a constant value or a set of values. The `#define` directive is processed by the preprocessor before the source code is compiled, and it replaces all occurrences of the macro with its corresponding value.

The basic syntax of the `#define` directive is:

```
#define MACRO_NAME value
```

where `MACRO_NAME` is the name of the macro, and `value` is the constant value it represents. For example:

```
#define PI 3.14
```

This defines the macro `PI` to have the value `3.14`. Any occurrences of `PI` in the code will be replaced with `3.14` by the preprocessor.

Macros can also be defined with parameters to create macro functions. The syntax is:

```
#define MACRO_NAME (param1, param2, ...) expression
```

where `param1`, `param2`, etc. are the parameters of the macro, and `expression` is the value that the macro returns. For example:

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))
```

This defines a macro function `MAX` that takes two parameters and returns the maximum of the two. The macro function is called by passing in two values, and the preprocessor replaces the call with the result of the expression.

Note that symbolic constants defined using `#define` do not have a data type, and they are not checked for type compatibility. Also, because the preprocessor replaces occurrences of the macro with its value, it is not possible to use `#define` to define variables.

2. Difference between `undef` and `ifndef`.

`#undef` and `#ifndef` are preprocessor directives in C and C++.

`#undef` is used to remove the definition of a previously defined macro. It undefines a symbol that has already been defined using `#define`. For example:

```
#define MAX_SIZE 100
```

```
#undef MAX_SIZE
```

`#ifndef` is used to check if a macro has not been defined. If the macro is not defined, the code within the following `#endif` will be compiled. For example:

```
#ifndef MAX_SIZE
```

```
#define MAX_SIZE 100
```

```
#endif
```

3. Write the Difference between Binary File and Text File?

S. No.	Text file	Binary File
1.	The text files can easily be transferred from one computer system to another.	Binary files cannot easily be transferred from one computer system to another due to variations in the internal representation which varies from computer to computer.
2.	It stores data using ASCII format i.e. human-readable graphic characters.	It stores data in binary format i.e. with the help of 0 and 1.
3.	These files are easily readable and modifiable because the content written in text files is human readable. Content written in binary files is not human-readable and looks like encrypted content.	These files are not easily readable and modifiable because the content written in binary files is not human-readable and it is encrypted content.
4.	These files create portability problems.	These files are easily portable.
5.	Text files save the data by converting each digit in data into ASCII format which will take up much of the space as compared to the required one. For example, the number 546378 is an integer that should occupy 4 bytes in the disk but it will occupy 6	These save memory because the data of any type will get stored in memory as per its memory size. For example, any integer number irrespective of individual digits in the number will be stored by consuming 4 bytes.

S. No.	Text file	Binary File
	bytes, 1 byte for each digit in the number.	
6.	Any file is by default text file.	The ios:: binary mode has to be used with binary files while opening them.
7.	Error in a textual file can be easily recognized and eliminated.	Error in a binary file corrupts the file and is not easily detected.
8.	In a text file, a new line character is first converted to a carriage return-line feed combination and then written to the disk. Vice versa happens when a line is read from the text file.	In binary file, no such conversion from newline to carriage return-line feed combination is done.
9.	In a text file, a special character with ASCII code 26 is inserted at the end of the file. This character signals the EOF to the program when encountered.	There is no such special character in the binary file to signal EOF.
10.	Text files are used to store data more user friendly.	Binary files are used to store data more compactly.
11.	Mostly .txt and .rtf are used as extensions to text files.	Can have any application defined extension.

4. Define different types of file input output functions?

There are several types of file input/output (I/O) functions in C and C++:

1. `fopen()` and `fclose()`: These functions are used to open and close a file. `fopen()` returns a file pointer to the opened file, which can then be used for reading or writing to the file. `fclose()` is used to close the file after you are done with it.
2. `fread()` and `fwrite()`: These functions are used to read and write binary data to and from a file. `fread()` is used to read a specified number of bytes from a binary file into memory, while `fwrite()` is used to write binary data to a file.
3. `fprintf()` and `fscanf()`: These functions are used to read and write formatted text to and from a file. `fprintf()` is used to write formatted text to a file, while `fscanf()` is used to read formatted text from a file.
4. `fgets()` and `fputs()`: These functions are used to read and write strings to and from a file. `fgets()` reads a string from a file and stores it in a character array, while `fputs()` writes a string to a file.
5. `getc()` and `putc()`: These functions are used to read and write individual characters to and from a file. `getc()` is used to read a single character from a file, while `putc()` is used to write a single character to a file.

5. Define different file modes of operation?

When opening a file using the `fopen()` function in C and C++, you can specify the mode of operation for the file. The following are the different file modes of operation:

1. `"r"`: Read mode. The file is opened for reading. If the file does not exist, the `fopen()` function returns a `NULL` pointer.
2. `"w"`: Write mode. The file is opened for writing. If the file exists, its contents are truncated to zero length. If the file does not exist, a new file is created.
3. `"a"`: Append mode. The file is opened for writing. If the file exists, new data is appended to the end of the file. If the file does not exist, a new file is created.
4. `"rb"`: Read binary mode. The file is opened for reading in binary mode. If the file does not exist, the `fopen()` function returns a `NULL` pointer.
5. `"wb"`: Write binary mode. The file is opened for writing in binary mode. If the file exists, its contents are truncated to zero length. If the file does not exist, a new file is created.

6. "ab": Append binary mode. The file is opened for writing in binary mode. If the file exists, new data is appended to the end of the file. If the file does not exist, a new file is created.

It's important to choose the correct mode of operation when opening a file, as it determines the behavior of the file I/O operations.

1) What are Preprocessor commands? List some Preprocessor commands with the example?

Preprocessor commands are special instructions that are processed by the preprocessor before the actual compilation of the code. They are indicated by the # symbol at the beginning of a line.

Some common preprocessor commands in C and C++ are:

1. #define: This directive is used to define a macro. For example:
`#define MAX_SIZE 100`
2. #include: This directive is used to include a header file in your code. For example:
`#include <stdio.h>`
3. #ifdef and #ifndef: These directives are used for conditional compilation. #ifdef checks if a macro is defined, and if it is, the code within the following #endif is compiled. #ifndef checks if a macro is not defined, and if it is not, the code within the following #endif is compiled. For example:
`#ifndef MAX_SIZE`
`#define MAX_SIZE 100`
`#endif`
4. #undef: This directive is used to undefine a previously defined macro. For example:
`#undef MAX_SIZE`
5. #error: This directive is used to generate a compiler error. For example:
`#error This is a compiler error.`

These are some of the commonly used preprocessor commands in C and C++. There are several other preprocessor commands as well, each with specific uses and applications.

2) Define a macro that finds the maximum of two numbers. Write a C program that uses the macro and prints the maximum of two numbers.

A macro is a fragment of code that is given a name and can be reused multiple times. In C and C++, you can define a macro using the `#define` preprocessor directive.

Here's an example of a macro that finds the maximum of two numbers:

```
#define MAX(x, y) ((x) > (y) ? (x) : (y))
```

This macro takes two arguments, `x` and `y`, and returns the greater of the two. The `((x) > (y) ? (x) : (y))` expression uses the ternary operator to determine the maximum value.

Here's a C program that uses the macro to print the maximum of two numbers:

```
#include <stdio.h>

#define MAX(x, y) ((x) > (y) ? (x) : (y))

int main()
{
    int x, y;
    printf("Enter two numbers: ");
    scanf("%d%d", &x, &y);
    printf("The maximum of %d and %d is %d\n", x, y, MAX(x, y));
    return 0;
}
```

This program first includes the standard input/output header file `<stdio.h>`. Then, it defines the `MAX` macro as described above. In the `main` function, it takes two numbers as input from the user, and then uses the `MAX` macro to determine the maximum of the two numbers, which is then printed to the screen.

3) Write about Positioning functions (fseek,rewind and ftell)

`fseek`, `rewind`, and `ftell` are functions in the C standard library for controlling the position of a file pointer in a stream. These functions are used to move the file pointer within a file, allowing you to read or write to different parts of the file.

1. `fseek`: This function allows you to move the file pointer to a specific position within a file. The function takes three arguments: the file stream, the number of bytes to move the pointer, and the origin of the move. The origin can be `SEEK_SET`, `SEEK_CUR`, or `SEEK_END`, which indicate the beginning of the file, the current position of the file pointer, and the end of the file, respectively. For example:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    FILE *fp;
```

```
    fp = fopen("file.txt", "r");
```

```
    fseek(fp, 10, SEEK_SET);
```

```
    // file pointer is now positioned at 10th byte of the file
```

```
    fclose(fp);
```

```
    return 0;
```

```
}
```

2. `rewind`: This function is used to reset the file pointer to the beginning of the file. It takes a single argument, the file stream, and sets the file pointer to the first byte in the file. For example:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    FILE *fp;
```

```
    fp = fopen("file.txt", "r");
```

```
    fseek(fp, 10, SEEK_SET);
```

```
    // file pointer is now positioned at 10th byte of the file
```

```

rewind(fp);
// file pointer is now positioned at the beginning of the file
fclose(fp);
return 0;
}

```

3. **ftell**: This function returns the current position of the file pointer in a file. It takes a single argument, the file stream, and returns the number of bytes from the beginning of the file to the current position of the file pointer. For example:

```

#include <stdio.h>

int main()
{
    FILE *fp;
    long int position;
    fp = fopen("file.txt", "r");
    fseek(fp, 10, SEEK_SET);
    // file pointer is now positioned at 10th byte of the file
    position = ftell(fp);
    printf("The file pointer is now at %ld\n", position);
    fclose(fp);
    return 0;
}

```

These functions allow you to control the position of the file pointer in a file and read or write to specific parts of the file. They are useful in various scenarios, such as when you want to skip a certain number of bytes in a file, or when you need to return to the beginning of a file after reading it.

4) Write a C program to count the number of times a character occurs in a text file. The file name and the character are supplied as command-line arguments.

Here is a sample program in C to count the number of times a character occurs in a text file:

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[])
{
    if (argc != 3)
    {
        printf("Usage: %s <file name> <character>\n", argv[0]);
        return 1;
    }
    FILE *fp;
    fp = fopen(argv[1], "r");
    if (fp == NULL)
    {
        printf("Error opening file\n");
        return 1;
    }
    int count = 0;
    char ch;
    while ((ch = fgetc(fp)) != EOF)
    {
        if (ch == argv[2][0])
        {
            count++;
        }
    }
}
```

```

    }
}
fclose(fp);

printf("The character '%c' occurs %d times in the file %s\n", argv[2][0],
count, argv[1]);

return 0;
}

```

The program takes two command-line arguments: the name of the file and the character to count. The file is opened in read-only mode using `fopen`, and a loop is used to read the file character by character using `fgetc`. If the current character is equal to the character to count, the `count` variable is incremented. The loop continues until the end of the file is reached. Finally, the file is closed using `fclose` and the count is printed to the screen.

5) Write about file status functions (error handling) with example?

File status functions are used to check the status of a file after it has been opened or after an input/output operation has been performed on the file. These functions are used to check for errors and handle them appropriately. Here are some commonly used file status functions in C and C++, along with examples:

1. `ferror()`: This function returns a non-zero value if an error has occurred on the file stream.

Example:

```

FILE *fp;

fp = fopen("file.txt", "r");
if (fp == NULL || ferror(fp))
{
    printf("Error opening file\n");
    return 1;
}

```

2. `clearerr()`: This function clears the error and end-of-file indicators for the specified file stream.

Example:

```
FILE *fp;
fp = fopen("file.txt", "r");
if (fp == NULL)
{
    printf("Error opening file\n");
    return 1;
}
if (ferror(fp))
{
    clearerr(fp);
    printf("Error cleared\n");
}
```

3. `feof()`: This function returns a non-zero value if the end-of-file indicator for the specified file stream is set.

Example:

```
FILE *fp;
fp = fopen("file.txt", "r");
if (fp == NULL)
{
    printf("Error opening file\n");
    return 1;
}
char ch;
while (!feof(fp))
```

```
{  
    ch = fgetc(fp);  
    printf("%c", ch);  
}  
fclose(fp);
```

In the example, `feof` is used to determine when the end of the file has been reached. The loop continues until the end of the file is reached, at which point `feof` will return a non-zero value and the loop will exit. The file is then closed using `fclose`.

It is important to check for errors and handle them appropriately when working with files, as this helps to ensure the integrity of the data and prevent unexpected behavior in your program.

Unit 4

1. What is a Function? Write about signature of a function?

A function is a self-contained block of code that performs a specific task and can be called from other parts of a program. Functions allow you to structure your code into reusable and organized blocks, making it easier to read, debug, and maintain.

The signature of a function is a unique identifier that includes the function name, the number and types of parameters the function takes, and the type of value the function returns (if any). The signature of a function must be declared before it is used in a program, and it serves to identify the function and ensure that it is called correctly.

Here is an example of a function signature in C:

```
int add(int x, int y);
```

In this example, the function is named `add`, takes two parameters of type `int`, and returns a value of type `int`. The signature is used to declare the function before it is defined. The definition of the function, which specifies the task the function will perform, must match the signature exactly.

It is important to note that the function signature only specifies the interface to the function and does not include the implementation details. The implementation details are specified in the function definition.

2. Write about limitation of recursion?

Recursion is a powerful concept in programming, but it has some limitations that must be considered when using it in a program. Here are some of the limitations of recursion:

1. **Maximum Recursion Depth:** All programming languages have a limited maximum recursion depth, which is the maximum number of times a recursive function can call itself before stopping. If a recursive function exceeds this limit, it will result in a stack overflow error.
2. **Performance:** Recursive algorithms can be slower than equivalent iterative algorithms, especially for large input sizes, because each recursive call creates a new stack frame and consumes memory.
3. **Code Complexity:** Recursive algorithms can be more complex to understand and debug than equivalent iterative algorithms, especially for non-trivial problems.
4. **Infinite Recursion:** If the base case of a recursive function is not defined correctly, the function can enter into an infinite loop, consuming all available memory and eventually crashing the program.
5. **Space Complexity:** Recursive algorithms consume more memory than equivalent iterative algorithms, especially for large input sizes, because of the creation of stack frames for each recursive call.

Therefore, while recursion can be a useful tool in certain situations, it should be used with caution and carefully considered in relation to other algorithms and data structures. In general, iterative solutions are preferred for problems where performance and space complexity are important considerations.

3. Explain passing parameters to functions.

Passing parameters to functions allows a function to receive input from the calling code and operate on that input. This input can be used to perform specific actions, return results, or modify data. There are two main ways to pass parameters to functions in C: pass by value and pass by reference.

1. **Pass by Value:** In pass by value, the function receives a copy of the argument passed in, rather than a reference to the actual argument. This means that any changes made to the argument within the function will not affect the original argument in the calling code.

Example:

```

#include <stdio.h>

void add_one(int x) {
    x++;
}

int main() {
    int num = 10;
    add_one(num);
    printf("%d\n", num); // Output: 10
    return 0;
}

```

2. **Pass by Reference:** In pass by reference, the function receives a reference to the actual argument passed in, rather than a copy. This means that any changes made to the argument within the function will affect the original argument in the calling code. This is achieved by passing a pointer to the argument.

Example:

```

#include <stdio.h>

void add_one(int *x) {
    (*x)++;
}

int main() {
    int num = 10;
    add_one(&num);
    printf("%d\n", num); // Output: 11
    return 0;
}

```

It is important to choose the appropriate method of passing parameters based on the requirements of the function and the desired behavior. In general, pass by value is used when the function does not need to modify

the original argument, while pass by reference is used when the function needs to modify the original argument.

4. Explain passing pointers to functions.

Passing pointers to functions allows a function to operate on data stored at a specific memory location. This is useful in cases where you need to modify data in place, or when you want to pass large amounts of data to a function without copying it.

Here's how you can pass a pointer to a function in C:

```
#include <stdio.h>

void increment(int *x) {
    (*x)++;
}

int main() {
    int num = 10;
    increment(&num);
    printf("%d\n", num); // Output: 11
    return 0;
}
```

In the example above, the `increment` function takes a pointer to an integer `x` as an argument. The `&` operator is used to pass the address of `num` to the function. Inside the function, the value stored at that memory location is incremented using the `++` operator.

Passing pointers to functions can be more efficient than passing values, because it eliminates the need to make a copy of the data. This can be particularly important when working with large data structures. Additionally, passing pointers allows functions to modify the original data, rather than just the copy.

It's important to remember that when passing a pointer to a function, you're also passing control over that memory location. The function can modify or even destroy the data stored at that location, so it's important to

make sure that the function is used responsibly and that the pointer is valid before passing it to the function.

5. Describe the allocation of memory for arrays of different data types

In C, arrays are allocated memory dynamically at runtime. The amount of memory required for an array depends on its data type and size.

1. Character arrays: Character arrays are stored in 1 byte of memory each. A character array of size n requires n bytes of memory.

Example:

```
char name[10];
```

The above code will allocate 10 bytes of memory for the character array `name`.

2. Integer arrays: Integers are stored in 2 or 4 bytes of memory, depending on the architecture of the system. On most systems, integers are stored in 4 bytes of memory. An integer array of size n requires $4n$ bytes of memory.

Example:

```
int numbers[10];
```

The above code will allocate 40 bytes of memory for the integer array `numbers`.

3. Float arrays: Floats are stored in 4 bytes of memory. A float array of size n requires $4n$ bytes of memory.

Example:

```
float decimals[10];
```

The above code will allocate 40 bytes of memory for the float array `decimals`.

4. Double arrays: Doubles are stored in 8 bytes of memory. A double array of size n requires $8n$ bytes of memory.

Example:

```
double prices[10];
```

The above code will allocate 80 bytes of memory for the double array `prices`.

It's important to note that the amount of memory required for arrays of different data types is different and should be considered when designing and writing code that makes use of arrays. Additionally, the amount of memory required for arrays of the same data type can vary based on the size of the array, the size of the data type, and the architecture of the system on which the code is running.

1. Write about standard functions with examples?

In C, there are several standard functions that are available for use in any C program. These functions are defined in the standard library and can be used without the need for any special libraries or headers.

1. `strlen`: This function is used to find the length of a string. It takes a string as an argument and returns the number of characters in the string, excluding the null terminator.

Example:

```
#include <string.h>
```

```
#include <stdio.h>
```

```
int main() {  
    char str[100] = "Hello, World!";  
    int len = strlen(str);  
    printf("The length of the string is: %d\n", len);  
    return 0;  
}
```

2. `strcpy`: This function is used to copy one string to another. It takes two arguments, the first being the destination string and the second being the source string.

Example:

```
#include <string.h>
#include <stdio.h>
int main() {
    char str1[100] = "Hello, World!";
    char str2[100];
    strcpy(str2, str1);
    printf("The destination string is: %s\n", str2);
    return 0;
}
```

3. `strcat`: This function is used to concatenate two strings. It takes two arguments, the first being the destination string and the second being the source string.

Example:

```
#include <string.h>
#include <stdio.h>
int main() {
    char str1[100] = "Hello, ";
    char str2[100] = "World!";
    strcat(str1, str2);
    printf("The concatenated string is: %s\n", str1);
    return 0;
}
```

4. `printf`: This function is used to output data to the screen. It takes a formatted string as an argument and can be used to output text, numbers, and other data.

Example:

```
#include <stdio.h>

int main() {
    int a = 10;
    float b = 12.5;
    printf("The value of a is: %d\n", a);
    printf("The value of b is: %f\n", b);
    return 0;
}
```

5. `scanf`: This function is used to input data from the keyboard. It takes a formatted string as an argument and can be used to input text, numbers, and other data.

Example:

```
#include <stdio.h>

int main() {
    int a;
    float b;
    printf("Enter the value of a: ");
    scanf("%d", &a);
    printf("Enter the value of b: ");
    scanf("%f", &b);
    printf("The value of a is: %d\n", a);
    printf("The value of b is: %f\n", b);
    return 0;
}
```

These are just a few examples of the standard functions available in C. There are many more functions available, each with its own specific use and application.

2. Explain about memory allocation functions with example.

Memory allocation functions are used to dynamically allocate memory in C. This allows you to allocate memory during runtime rather than at compile-time, which makes it possible to create arrays, data structures, and other objects with an unknown size beforehand.

Here are the two main memory allocation functions in C:

1. `malloc()`: The `malloc()` function is used to allocate a block of memory of a specified size. The size is specified in bytes. The function returns a pointer to the first byte of the newly allocated memory block, or a null pointer if the allocation failed.

Example:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n, i;
    int *p;
    printf("Enter the number of elements: ");
    scanf("%d", &n);
    // Allocating memory for n integers
    p = (int *)malloc(n * sizeof(int));
    if (p == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }
}
```

```

// Storing elements in the newly allocated memory
for (i = 0; i < n; i++)
{
    printf("Enter element %d: ", i + 1);
    scanf("%d", &p[i]);
}
// Printing the elements
printf("Elements in the array:\n");
for (i = 0; i < n; i++)
{
    printf("%d\n", p[i]);
}
// Freeing the memory
free(p);
return 0;
}

```

2. `calloc()`: The `calloc()` function is used to allocate a block of memory of a specified size and initialize it to zero. The size is specified in terms of the number of elements, not bytes. The function returns a pointer to the first byte of the newly allocated memory block, or a null pointer if the allocation failed.

Example:

```

#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n, i;
    int *p;

```

```

printf("Enter the number of elements: ");
scanf("%d", &n);

// Allocating memory for n integers and initializing to zero
p = (int *)calloc(n, sizeof(int));
if (p == NULL)
{
    printf("Memory allocation failed\n");
    return 1;
}

// Printing the elements
printf("Elements in the array:\n");
for (i = 0; i < n; i++)
{
    printf("%d\n", p[i]);
}

// Freeing the memory
free(p);
return 0;
}

```

Note: The memory allocated by the memory allocation functions should be freed using the `free()` function when it's no longer needed to avoid memory leaks.

3.) What is recursion? Write a program for factorial using recursion?

Recursion is a programming technique in which a function calls itself to solve a problem. It is used when a problem can be broken down into smaller subproblems, which can then be solved recursively. The function stops calling itself when it reaches a base case, which is a simple case that can be solved without further recursion.

Here's an example of a C program to find the factorial of a number using recursion

```
#include <stdio.h>

int factorial(int n)
{
    if (n == 0)
        return 1;
    else
        return n * factorial(n-1);
}

int main()
{
    int num, result;
    printf("Enter a positive integer: ");
    scanf("%d", &num);
    result = factorial(num);
    printf("Factorial of %d = %d", num, result);
    return 0;
}
```

In this example, the `factorial` function takes an integer `n` as an argument and returns its factorial. If `n` is 0, the function returns 1, which is the base case. Otherwise, the function calls itself with `n-1` as an argument and returns the result multiplied by `n`. The main function reads an integer from the user, calls the `factorial` function, and prints the result.

4. Difference between call by value and call by reference.

In C programming, there are two methods of passing arguments to a function: call by value and call by reference.

Call by value: In this method, a copy of the actual argument is made and passed to the function. Any changes made to the argument inside the function have no effect on the original argument outside the function.

Example:

```
#include <stdio.h>

void swap(int x, int y)
{
    int temp;
    temp = x;
    x = y;
    y = temp;
}

int main()
{
    int a = 10, b = 20;
    printf("Before swapping: a = %d, b = %d\n", a, b);
    swap(a, b);
    printf("After swapping: a = %d, b = %d\n", a, b);
    return 0;
}
```

Output:

Before swapping: a = 10, b = 20

After swapping: a = 10, b = 20

Call by reference: In this method, a reference to the actual argument is passed to the function. Any changes made to the argument inside the function will affect the original argument outside the function.

Example:

```

#include <stdio.h>

void swap(int *x, int *y)
{
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}

int main()
{
    int a = 10, b = 20;
    printf("Before swapping: a = %d, b = %d\n", a, b);
    swap(&a, &b);
    printf("After swapping: a = %d, b = %d\n", a, b);
    return 0;
}

```

Output:

Before swapping: a = 10, b = 20

After swapping: a = 20, b = 10

In this example, the `swap` function takes two pointers to integers as arguments. The `swap` function swaps the values stored at the memory addresses pointed to by the arguments. The main function declares two integer variables `a` and `b`, and passes their addresses to the `swap` function using the address-of operator `&`.

5. Write a recursive function for Fibonacci series.

Let's see the fibonacci series program in c using recursion

```

#include<stdio.h>

void printFibonacci(int n){
    static int n1=0,n2=1,n3;

```

```

    if(n>0){
        n3 = n1 + n2;
        n1 = n2;
        n2 = n3;
        printf("%d ",n3);
        printFibonacci(n-1);
    }
}

int main(){
    int n;
    printf("Enter the number of elements: ");
    scanf("%d",&n);
    printf("Fibonacci Series: ");
    printf("%d %d ",0,1);
    printFibonacci(n-2);//n-
2 because 2 numbers are already printed
    return 0;
}

```

Output:

```

Enter the number of elements:15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

```

Unit 5

1. Define order of complexity.

Order of complexity is a term used to describe the efficiency of an algorithm in terms of the amount of time or space required to solve a problem. The order of complexity is expressed using Big O notation and provides an upper bound on the growth rate of the running time or memory usage of an algorithm as the size of the input increases.

The most commonly used orders of complexity include:

- $O(1)$, also known as constant time, which indicates that the time or space required by the algorithm remains the same regardless of the size of the input.

- $O(\log n)$, which represents a logarithmic time complexity and indicates that the time or space required by the algorithm increases logarithmically with the size of the input.
- $O(n)$, which represents a linear time complexity and indicates that the time or space required by the algorithm increases linearly with the size of the input.
- $O(n \log n)$, which represents a log-linear time complexity and indicates that the time required by the algorithm increases as the product of the size of the input and the logarithm of the size of the input.
- $O(n^2)$, which represents a quadratic time complexity and indicates that the time or space required by the algorithm increases as the square of the size of the input.
- $O(2^n)$, which represents an exponential time complexity and indicates that the time required by the algorithm doubles with each additional unit of the size of the input.

The goal of algorithm analysis is to determine the order of complexity of an algorithm, so that its efficiency can be compared with other algorithms and the best algorithm can be selected for a particular problem.

2. What is an algorithm and what are its advantages?

An algorithm is a set of instructions that define a process to solve a problem. It is a step-by-step procedure for solving a specific problem, and it is a crucial part of computer science and software engineering.

In C, algorithms are used to solve problems efficiently by making use of the language's features and libraries. There are several advantages to using algorithms in C, including:

1. Better performance: Algorithms can improve the performance of a program by reducing the time and space complexity.
2. Reusability: Algorithms can be written once and used multiple times, making it easier to write programs and reducing the amount of code needed to solve a problem.
3. Abstraction: Algorithms provide an abstraction layer, hiding the underlying implementation details and making it easier to understand and modify the code.
4. Debugging: Algorithms can be easily debugged and tested, allowing for easier detection and correction of bugs.

5. Portability: Algorithms written in C can be easily ported to other programming languages and platforms, making it easier to write cross-platform code.

Overall, algorithms play a crucial role in solving problems efficiently and effectively in C and other programming languages. They are used in a wide range of applications, including data analysis, image processing, and machine learning, among others.

3.) Write an algorithm to check whether a given number is prime or not.

ep 1: Start
Step 2: Read number n
Step 3: Set f=0
Step 4: For i=2 to n-1
Step 5: If n mod i=0 then
Step 6: Set f=1 and break
Step 7: Loop
Step 8: If f=0 then
 print 'The given number is prime'
 else
 print 'The given number is not prime'
Step 9: Stop

Program code

```
#include<stdio.h>

#include<conio.h>

void main()

{

clrscr();

int n,i,f=0;

printf("Enter the number: ");

scanf("%d",&n);

for(i=2;i<n;i++)
```

```

{
    if(n%i==0)
    {
        f=1;
        break;
    }
}
if(f==0)
    printf("The given number is prime");
else
    printf("The given number is not prime");
getch();
}

```

Output

Enter the number : 5

The given number is prime

4. Differences between linear search and binary search.

Basis of comparison	Linear search	Binary search
Definition	The linear search starts searching from the first element and compares each element with a searched element till the element is not found.	It finds the position of the searched element by finding the middle element of the array.

Sorted data	In a linear search, the elements don't need to be arranged in sorted order.	The pre-condition for the binary search is that the elements must be arranged in a sorted order.
Implementation	The linear search can be implemented on any linear data structure such as an array, linked list, etc.	The implementation of binary search is limited as it can be implemented only on those data structures that have two-way traversal.
Approach	It is based on the sequential approach.	It is based on the divide and conquer approach.
Size	It is preferable for the small-sized data sets.	It is preferable for the large-size data sets.
Efficiency	It is less efficient in the case of large-size data sets.	It is more efficient in the case of large-size data sets.
Worst-case scenario	In a linear search, the worst-case scenario for finding the element is $O(n)$.	In a binary search, the worst-case scenario for finding the element is $O(\log_2 n)$.
Best-case scenario	In a linear search, the best-case scenario for finding the first element in the list is $O(1)$.	In a binary search, the best-case scenario for finding the first element in the list is $O(1)$.
Dimensional array	It can be implemented on both a single and multidimensional array.	It can be implemented only on a multidimensional array.

5. Compare different sorting algorithms.

There are many sorting algorithms, each with its own strengths and weaknesses. Here is a comparison of some common sorting algorithms:

1. Bubble sort: Bubble sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order. The algorithm gets its name from the way smaller elements "bubble" to the top of the list. Bubble sort has a time complexity of $O(n^2)$ in the worst case and is generally not used for large data sets.

2. Insertion sort: Insertion sort is similar to bubble sort, but with a key difference that it sorts the list as it iterates over it. The algorithm keeps the sorted portion of the list to the left of the current position and repeatedly inserts the next unsorted element into the correct place in the sorted portion of the list. Insertion sort has a time complexity of $O(n^2)$ in the worst case, but performs better than bubble sort for small data sets or nearly sorted data.
3. Selection sort: Selection sort is another simple sorting algorithm that works by repeatedly finding the minimum element from the unsorted portion of the list and swapping it with the first element in the unsorted portion. The time complexity of selection sort is $O(n^2)$ in the worst case, making it less efficient than more advanced algorithms for large data sets.
4. Quick sort: Quick sort is a divide-and-conquer algorithm that works by selecting a pivot element from the array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. The sub-arrays are then sorted recursively. Quick sort has an average time complexity of $O(n \log n)$ and is one of the fastest sorting algorithms for large data sets.
5. Merge sort: Merge sort is another divide-and-conquer algorithm that works by dividing the unsorted list into n sub-lists, each containing one element, and then repeatedly merging sub-lists to produce new, sorted sub-lists until there is only one sub-list remaining. Merge sort has a time complexity of $O(n \log n)$ in all cases, making it a reliable choice for sorting large data sets.
6. Heap sort: Heap sort is a comparison-based sorting algorithm that works by building a max-heap from the input data, and then repeatedly swapping the maximum element with the last element in the heap and reducing the size of the heap by one. Heap sort has a time complexity of $O(n \log n)$ and is efficient for both small and large data sets.

In conclusion, the choice of sorting algorithm will depend on the size and nature of the data set, and the desired time and space complexity.

2) Explain the algorithm for finding roots of a quadratic equations

Algorithm to find all the roots of a quadratic equation

1. Input the value of a , b , c .
2. Calculate $k = b^2 - 4ac$
3. If ($d < 0$)

Display "Roots are Imaginary, calculator1 = $(-b + i\sqrt{k}) / 2a$ and $r2 = (-b - i\sqrt{k}) / 2a$.

else if ($d = 0$)

Display "Roots are Equal" and calculate $r1 = r2 = (-b / 2*a)$

else

Display "Roots are real and calculate $r1 = -b + \sqrt{d} / 2*a$ and $r2 = -b - \sqrt{d} / 2*a$

4. Print $r1$ and $r2$.

5. End the algorithm.

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main()
```

```
{
```

```
double a, b, c, discriminant, root1, root2, realPart, imaginaryPart;
```

```
printf("Enter coefficients a, b and c: ");
```

```
scanf("%lf %lf %lf",&a, &b, &c);
```

```
discriminant = b*b-4*a*c;
```

```
// condition for real and different roots
```

```
if (discriminant > 0)
```

```
{
```

```
// sqrt() function returns square root
```

```
root1 = (-b+sqrt(discriminant))/(2*a);
```

```
root2 = (-b-sqrt(discriminant))/(2*a);
```

```
printf("root1 = %.2lf and root2 = %.2lf",root1 , root2);
```

```
}
```

```

//condition for real and equal roots
else if (discriminant == 0)
{
root1 = root2 = -b/(2*a);
printf("root1 = root2 = %.2lf;", root1);
}
// if roots are not real
else
{
realPart = -b/(2*a);
imaginaryPart = sqrt(-discriminant)/(2*a);
printf("root1 = %.2lf+%.2lfi and root2 = %.2f-%.2fi", realPart,
imaginaryPart, realPart, imaginaryPart);
}
return 0;
}

```

Input-

Enter coefficients a, b and c: 1 2 3

Output

Root1=-1+1.41421 and root2=1+1.41421

4)

Explain Bubble sort algorithm with its order of complexity

- The bubble sort is also known as the ripple sort.
- The bubble sort is probably the first, reasonably complex module that any beginning programmer has to write.
- It is a very simple construct which introduces the student to the fundamentals of how sorting works.
- A bubble sort makes use of an array and some sort of "swapping" mechanism. Most programming languages have a built-in function to swap elements of an array.

- Even if a swapping function does not exist, only a couple of extra lines of code are required to store one array element in a temporary field in order to swap a second element into its place.
- Then the first element is moved out of the temporary field and back into the array at the second element's position.
- Here is a simple example of how a bubble sort works: Suppose you have a row of children's toy blocks with letters on them.
- They are in random order and you wish to arrange them in alphabetical order from left to right.
- Example:

Step 1. Begin with the first block. In this case, the letter G. (Fig. 1.)



Fig. 1

Step 2. Look at the block just to the right of it.

Step 3. If the block to the right should come before the block on the left, swap them so that they are in order (Fig. 2.)



Fig. 2

If you were doing this by hand, you might just pick the blocks to be moved with one in each hand and cross your arms to swap them. Or you might move the first one out of its position temporarily, move the second one in its place, then move the first one to the now empty position (this is the difference between having a single function to do the swap, or writing some code to do it).

Step 4. Compare the next block in line with the first, and repeat step 3. Do this until you run out of blocks. Then begin step one again with the second block. (Fig. 3,4,5,6)

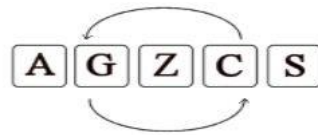


Fig. 3 - Pass #2

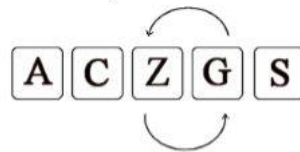


Fig. 4 - Pass #3



Fig. 5 - Pass #4



Fig. 6 - Completed Sort

The bubble sort gets its name because elements tend to move up into the correct order like bubbles rising to the surface.

Deriving the time complexity of the bubble sort algorithm as follows:

- According to above algorithm our time complexity equation formed is:

Worst case:

$$T(n) = C1 * n + C2 * (n-1)(n-i+1) + C3 * (n-1)(n-i) + C4 * (n-1)(n-i)$$

i is 1,2,3,4,5.....,n i.e i iterator is increasing

$$\begin{aligned} &= C1 * n + C2 * (nn - in + n - n - i + 1) + C3(nn - ni - n - i) + C4 * (nn - n * i - n - i) \\ &= C1n + C2(nn - in - i + 1) + C3(nn - ni - n - i) + C4 * (nn - n * i - n - i) \\ &= C1n + C2(nn) - C2(in) - C2i + C2 + C3(nn) - C3(ni) - C3n - C3i + C4(nn) - C4(ni) - C4n - C4 * i \\ &= n(C1 - C3 - C4) + nn(C2 + C3 + C4) + ni(-C2 - C3 - C4) + i * (-C2 - C3 - C4) + C2 \\ &= O(n * n) \\ &= O(n^2) \end{aligned}$$

Best case:

For sorted array 3rd line in bubble sort algorithm is not satisfied that's why of course 4th line will also not execute.

$$T(n) = C1n + C2 * ((n-1)(n-i+1)) + C3((n-1)(n-i)) + C4 * ((n-1)(n-i))$$

So that's why iterating one time that is till n times putting n in place of i in the equation we get:

$$\begin{aligned}
T(n) &= C_1n + C_2 ((n-1)(n-n+1)) + C_3*((n-1)(n-n)) + C_4 * 0 \\
&= C_1n + C_2 (n-1) + C_3*0 + C_3*0 + C_4*0 \\
&= C_1n + C_2n - C_2 \\
&= n*(C_1+C_2) - C_2 \\
&= O(n)
\end{aligned}$$

5) Explain the techniques of Selection sort and Insertion sort with the help of an example array.

Sorting

- Sorting is the process of arranging the elements of an array either in ascending or descending order.
- To do this various sorting techniques or sorting algorithms are used.
- Here, we see Bubble Sort, Insertion Sort, Selection Sort, Quick Sort, and Merge Sort in detail.

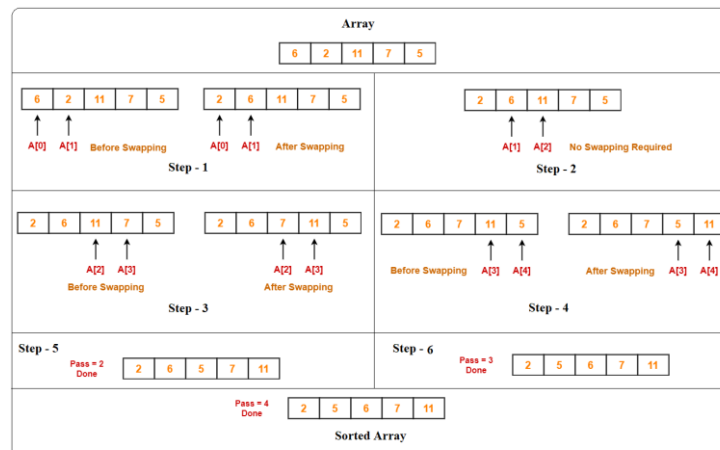
a) Bubble Sort

- Bubble sort is the simplest sorting technique among all other sortings.
- It is a stable, in-place sorting algorithm that uses no auxiliary data structures (extra space) while sorting.
- It works on the repeatedly swapping of adjacent elements until they are not in the intended order.
- This moves the largest element to the highest index of the array.
- To do this it uses multiple passes (scans) through an array.
- In each pass, bubble sort compares the adjacent elements of the array.
- It then swaps the two elements if they are in the wrong order.
- In each pass, bubble sort places the next largest element to its proper position.
- In short, it bubbles down the largest element to its correct position.
- The performance of bubble sort is poor in the real world because it is not suitable for large data sets.

Time Complexity:

- Best Case - $O(n)$
- Average Case - $O(n^2)$
- Worst Case - $O(n^2)$

Space Complexity - $O(1)$ (1)
 Example of Bubble Sort



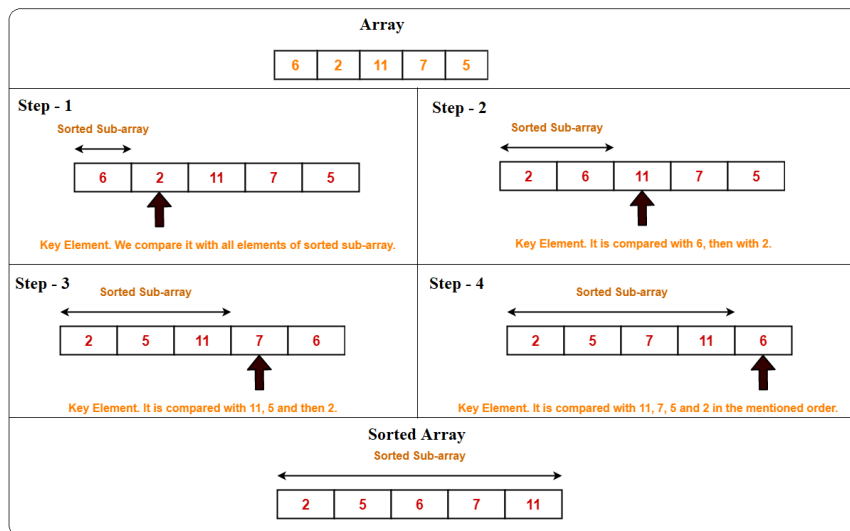
b] Insertion Sort

- According to the name insertion sort inserts each element of the array to its proper place.
- Insertion sort is an in-place sorting algorithm which not use auxiliary data structures while sorting.
- Insertion sort works similar to the sorting of playing cards in hands.
- It is assumed that the first element of the array is already sorted, and then it selects unsorted elements in the array.
- If the selected unsorted element is greater than the first element, it will be placed on the right side; otherwise, it will be placed on the left side.
- Similarly, all unsorted elements are taken and put in their exact position.
- Insertion sort is less efficient than the other sorting algorithms because it is not appropriate for large data sets.

Time Complexity:

- Best Case - $O(n)$
- Average Case - $O(n^2)$ (2)
- Worst Case - $O(n^2)$ (2)

Space Complexity – $O(1)$ (1)
 Example of Insertion Sort



c] Selection Sort

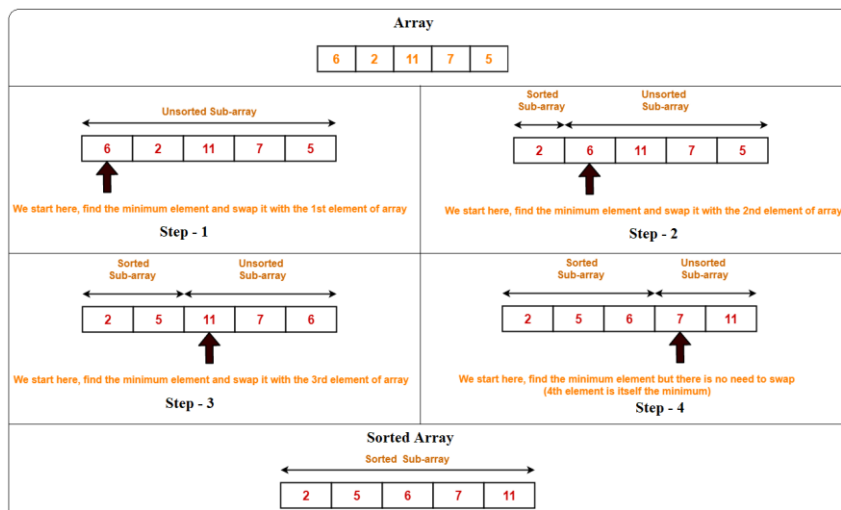
- Selection sort is one of the easiest approaches to sorting which works similarly as we sort out things in our day-to-day life.
- It is an in-place sorting algorithm because it uses no auxiliary data structures while sorting.
- In this algorithm, the array is divided into two parts, the first is the sorted part, and another one is the unsorted part.
- Initially, the sorted part of the array is empty, and the unsorted part is the given array.
- The sorted part is placed at the left, while the unsorted part is placed at the right.
- Selection sort finds the smallest element in the array and places it in the first place on the list.
- Then it finds the second smallest element in the array and places it in the second place.
- This process continues until all the elements are moved to their correct ordering.

Time Complexity:

- Best Case - $O(n^2)$
- Average Case - $O(n^2)$
- Worst Case - $O(n^2)$

Space Complexity - $O(1)$

Example of Selection Sort:



d] Quick Sort

- Quick sort is the very well-known, widely used, and most optimized sorting algorithm.
- This algorithm follows the divide and conquer approach.
- Divide and conquer is a technique of breaking down the algorithms into subproblems, then solving the subproblems, and combining the results back together to solve the original problem.
- Quick Sort follows a recursive algorithm.
- It divides the given array into two sections using a partitioning element called a pivot.
- The division performed is such that
 - All the elements to the left side of the pivot are smaller than the pivot.
 - All the elements to the right side of the pivot are greater than the pivot.
- After dividing the array into two sections, the pivot is set at its correct position.
- Then, sub arrays are sorted separately by applying a quick sort algorithm recursively in the same way.
- Finally, combine the already sorted array.
- Picking a good pivot is necessary for the fast implementation of quicksort.
- Some of the ways of choosing a pivot are as follows:
 - Pivot can be random, i.e. select the random pivot from the given array.
 - Pivot can either be the rightmost element or the leftmost element of the given array.
 - Select median as the pivot element.

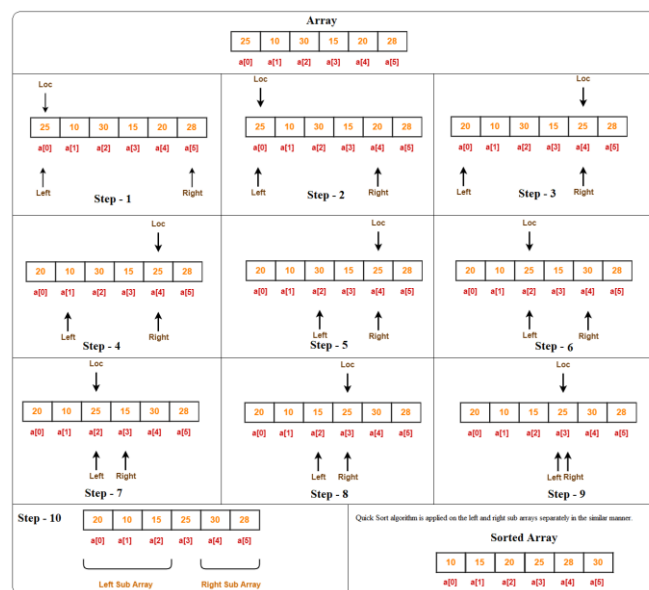
- Quick sort is an in-place sort, so it requires no temporary memory.
- Quick Sort is typically faster than other algorithms.
- Quick Sort tends to make excellent usages of the memory hierarchy like virtual memory or caches.
- Quick Sort can be easily parallelized due to its divide and conquer nature.
- It is not a stable sort i.e. the order of equal elements may not be preserved.

Time Complexity:

- Best Case - $O(n \log n)$
- Average Case - $O(n \log n)$
- Worst Case - $O(n^2)$

Space Complexity - $O(n \log n)$

Example of Quick Sort:



e] Merge Sort

- Merge sort is similar to the quick sort algorithm because it also uses the divide and conquers approach to sort the elements.
- It is one of the most popular and efficient sorting algorithms.
- Merge sort is a stable sorting algorithm.
- But it is not an in-place sorting algorithm because it uses additional memory for left and right sub-arrays.
- It divides the given array into two equal halves, which are the left sub-array and right sub-array.
- Perform merge sorting on each sub-array recursively.

